МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Реализация численных алгоритмов

Студент
гр. БИН-26-1 _____ Я. К. Берестнев

Преподаватель _____ М. В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Наибольший общий делитель

В первом задании пользователь вводит два целых числа a и b на отдельных строках. Если хотя бы одно число отрицательное или оба числа равны нулю, вывести Ошибка.

Для остальных значений найти наибольший общий делитель алгоритмом Евклида.

НОД - наибольшее положительное целое число, на которое оба исходных числа делятся без остатка. Для положительного a значение НОД(a, 0) равно a.

Алгоритм Евклида использует равенство НОД(a, b) = НОД(b, a mod b) при b > 0. Пару чисел последовательно заменяют по этому правилу, пока второе число не станет нулем. Первое число полученной пары является результатом

Чтобы решить задачу стоит обратиться к блок-схеме на рисунке 1

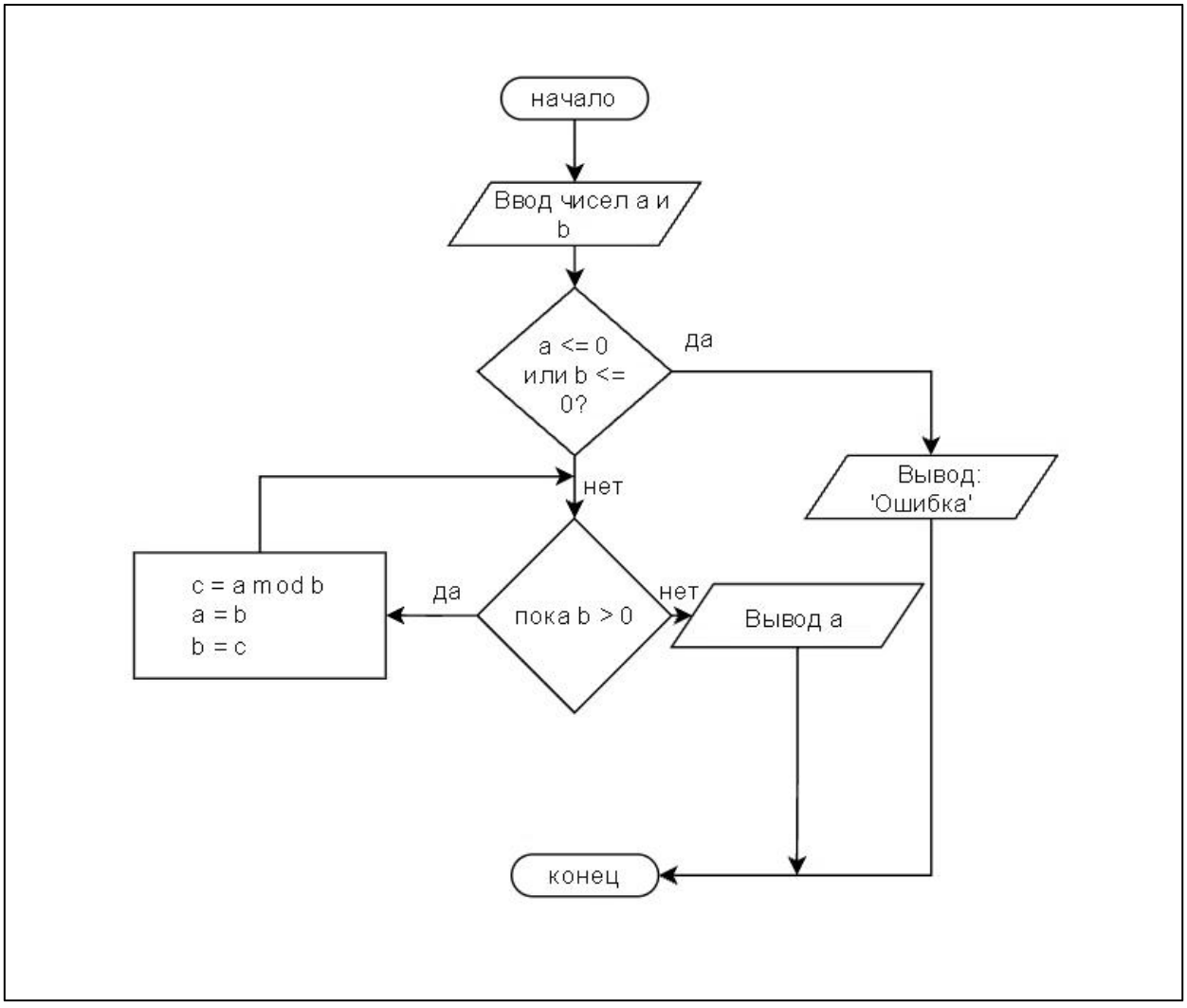


Рисунок 1 - Блок-схема для задания 1

Программа для решения первого задания представлена на рисунке 2

```
a = int(input())
b = int(input())
if a <= 0 or b <= 0:
    print('Ошибка')
else:
    while b > 0:
        c = a % b
        a = b
        b = c
    print(f'НОД: {a}')
```

Рисунок 2 – программа для первого задания

Результат при вводе 84 и 30 выводится НОД: 6.

1.2 Задание 2. Проверка числа на простоту

Во втором задании пользователь вводит целое число n.

Вывести Простое или Не простое. Простое число - целое число больше 1, которое имеет ровно два положительных делителя: 1 и само число. Все значения меньше 2 дают результат Не простое.

Для проверки перебрать возможные делители начиная с 2. Проверку завершить при нахождении делителя или когда квадрат проверяемого делителя превысит n.

Такая граница достаточна: если n = a * b, то хотя бы один из множителей a и b не превосходит квадратного корня из n

Чтобы решить задачу стоит обратиться к блок-схеме на рисунке 3

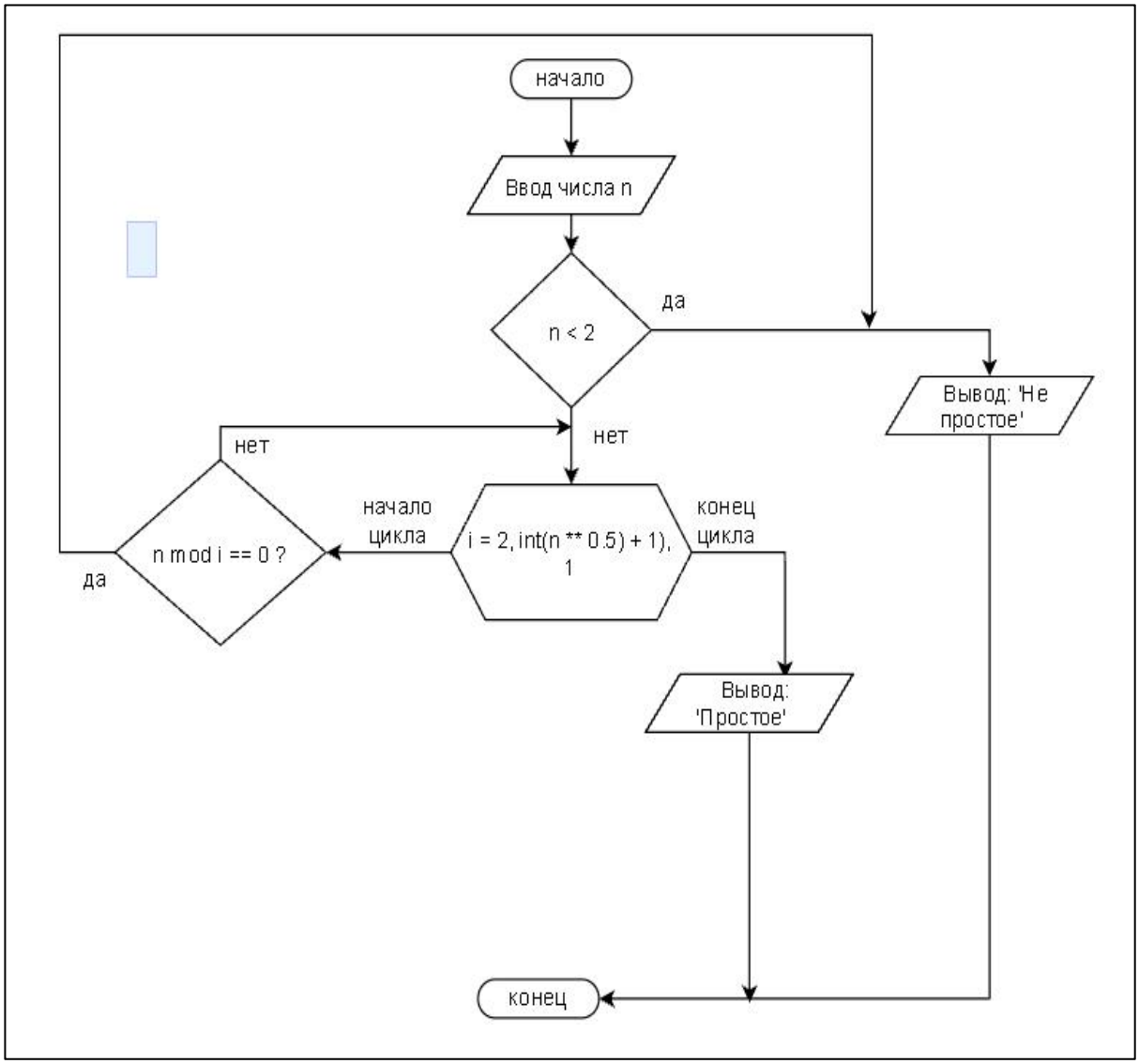


Рисунок 3 – Блок-схема для задания 2

Программа для решения второго задания представлена на рисунке 4.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
n = int(input())
if is_prime(n):
    print("Простое")
else:
    print("Не простое")
```

Рисунок 4 – программа для второго задания

Результат при вводе 49 число не простое.

1.3 Задание 3. Разложение на простые множители

В третьем задании пользователь вводит целое число n. Если n < 2, вывести Ошибка.

Разложить допустимое число на простые множители и вывести их в порядке неубывания на одной строке через один пробел, без пробела в конце. Повторяющиеся множители выводить столько раз, сколько они входят в разложение.

Разложение представляет исходное число как произведение простых чисел. Например, 18 = 2 * 3 * 3

Использовать последовательное деление: перебирать делители от 2, при делении без остатка уменьшать оставшееся число и повторно проверять тот же делитель. Продолжать до полного разложения

Чтобы решить задачу стоит обратиться к блок-схеме на рисунке 5

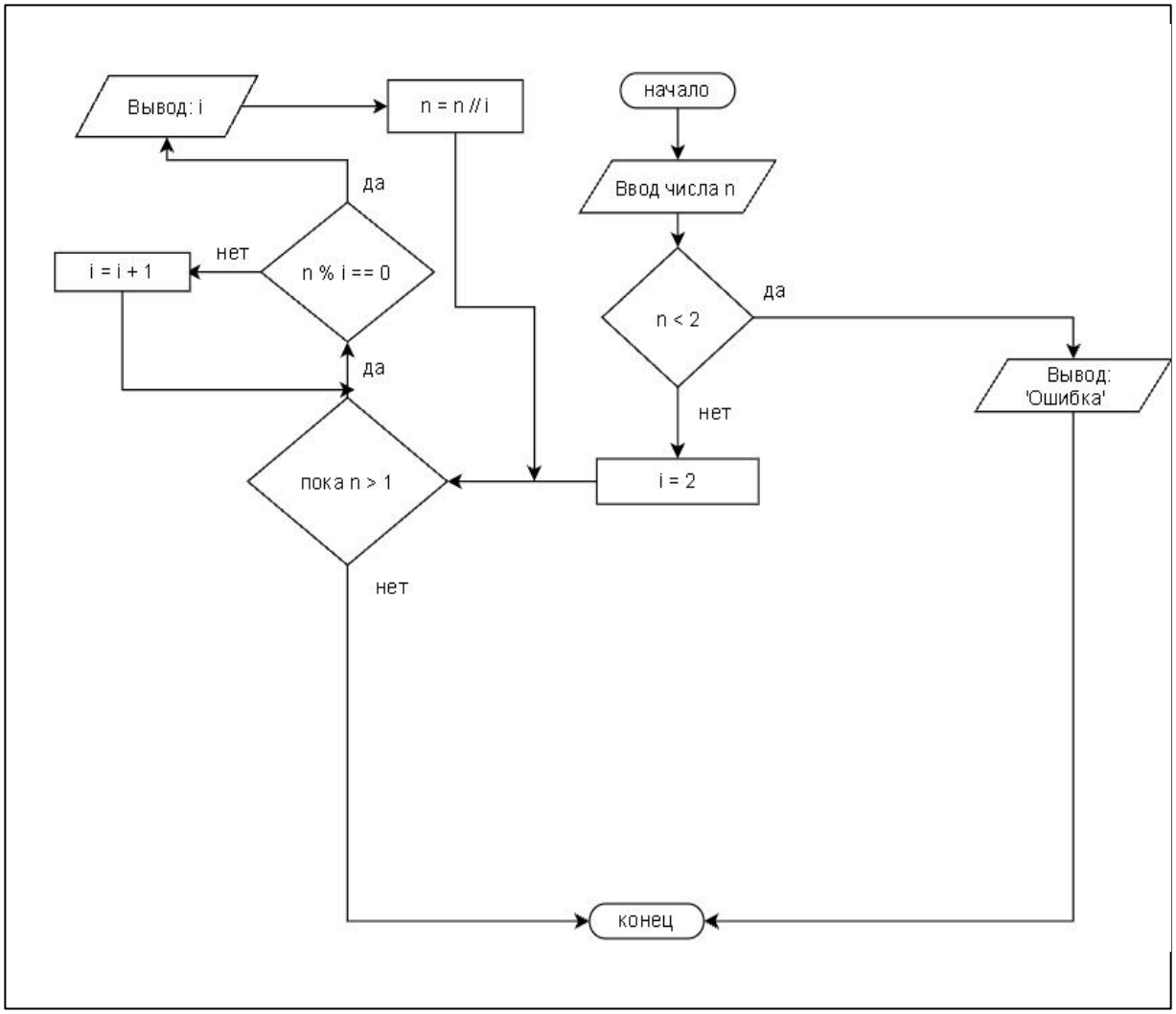


Рисунок 5 – Блок-схема для задания 3

Программа для решения третьего задания представлена на рисунке 6.

```
n = int(input())

if n < 2:
    print("Ошибка")
else:
    i = 2
    while n > 1:
        if n % i == 0:
            print(i, end=" ")
            n = n // i
        else:
            i += 1
```

Рисунок 6 – программа для третьего задания

Результат при вводе 60 получаем множители: 2,2,3,5

1.4 Задание 4. Цифры числа

В четвертом задании пользователь вводит целое число n. Если число отрицательное, вывести Ошибка.

Для допустимого значения определить количество цифр и их сумму, последовательно выделяя последнюю цифру остатком от деления на 10 и удаляя ее целочисленным делением на 10.

Работать с числовым значением, без преобразования числа в строку. У числа 0 одна цифра и сумма цифр равна 0. Ведущие нули исходной записи не учитываются

Чтобы решить задачу стоит обратиться к блок-схеме на рисунке 7.

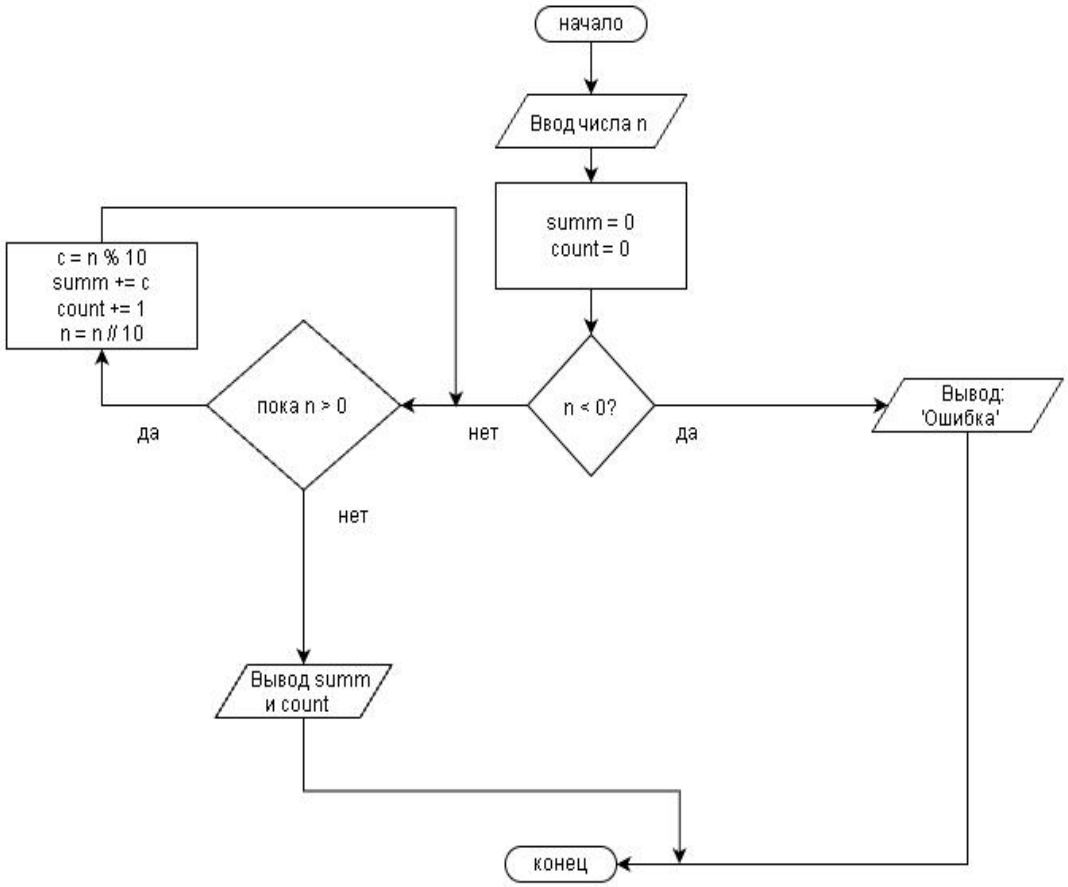


Рисунок 7 – Блок-схема для задания 4

Программа для решения четвертого задания представлена на рисунке 8.

```
n = int(input())
summ = 0
count = 0
if n < 0:
    print('Ошибка')
else:
    while n > 0:
        if n % 10:
            c = n % 10
            summ += c
        count += 1
        n = n // 10
print(count, summ)
```

Рисунок 8 – программа для четвертого задания

Результат при вводе 4050 получаем множители: кол-во цифр - 4 и сумму 9

1.5 Задание 5. Таблица расстояний

В задании 5 пользователь вводит целое число n от 1 до 10 включительно. При выходе за этот диапазон вывести Ошибка.

На прямой расположены точки с целочисленными координатами от 0 до n - 1. Построить таблицу расстояний между каждой парой точек.

В строке с номером i и столбце с номером j должно находиться расстояние между точками i и j. Оно равно большей координате минус меньшая.

Использовать вложенные циклы и условие для выбора порядка вычитания. Строки и столбцы выводить в порядке возрастания координат, без заголовков. Элементы строки разделять одним пробелом без пробела в конце.

Чтобы решить задачу стоит обратиться к блок-схеме на рисунке 9

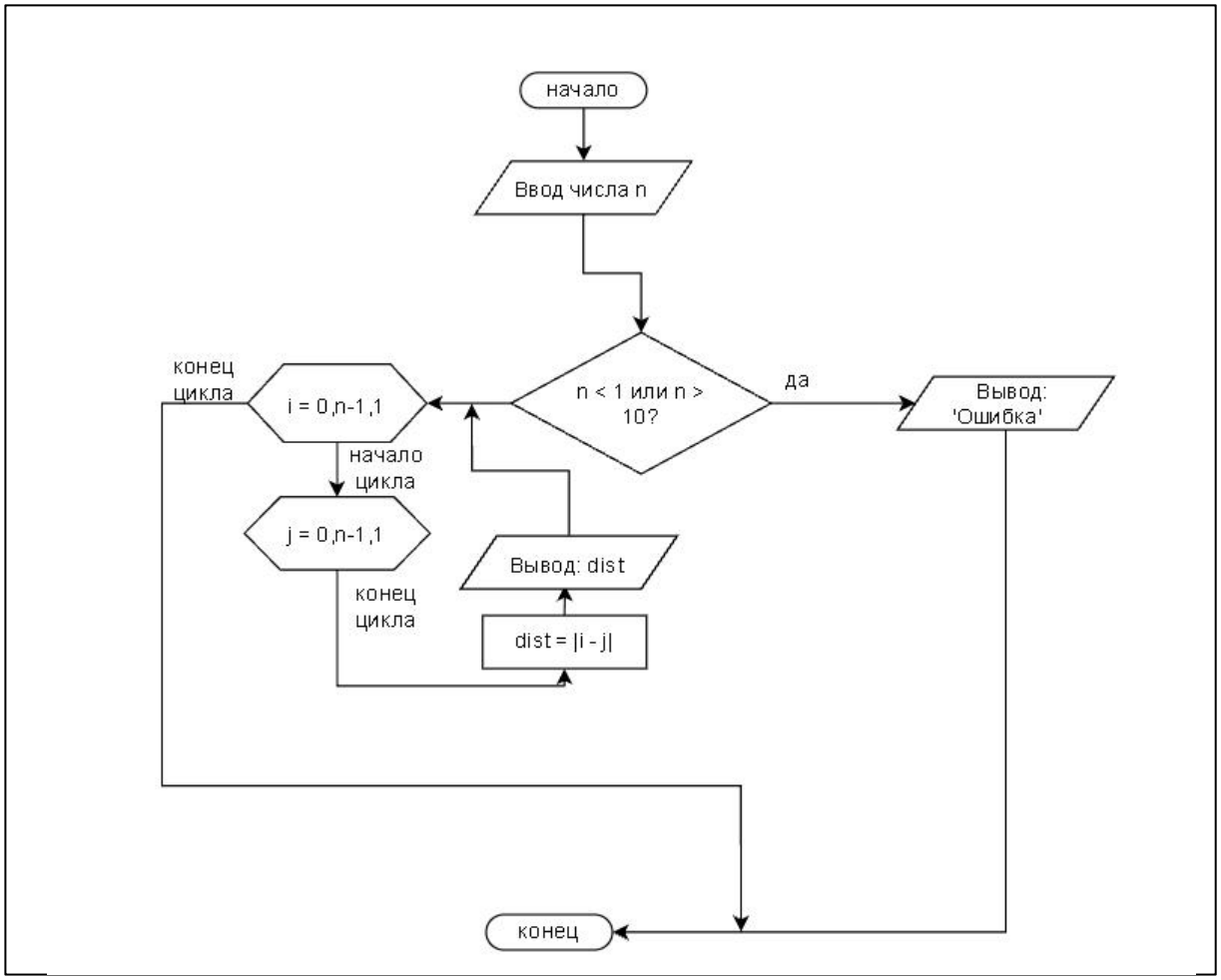


Рисунок 9 – Блок-схема для задания 5

Программа для решения пятого задания представлена на рисунке 10.

```
n = int(input())
if n < 1 or n > 10:
    print("Ошибка")
else:
    for i in range(n):
        for j in range(n):
            print(abs(i - j), end=" ")
        print()
```

Рисунок 8 – программа для пятого задания

Результат при вводе 4 мы получаем расстояние между точками